

Real-Time Active Noise Control System Using NLMS and FxNLMS

EE433 Final Project Report

Didem Ugur – 2375962

Baris Akdag – 2574457

June 2026

Abstract

This project implements an active noise control (ANC) system that attenuates low-frequency acoustic disturbances by generating an adaptive anti-noise signal. In the first stage, the ANC algorithms were designed and validated offline in MATLAB using normalized least mean squares (NLMS) and filtered-X normalized least mean squares (FxNLMS). In the final stage, the system was moved to a real-time prototype using LabVIEW for the user interface and audio visualization, and a C dynamic-link library (DLL) for low-level audio acquisition and sample-by-sample adaptive filtering. The implemented C engine supports two operating modes: a direct NLMS update for the basic feedforward case and an FxNLMS update that compensates for the secondary acoustic path. The real-time prototype uses a double-buffered Windows audio input stream, a 64-tap adaptive controller, a 128-tap estimated secondary path, and front-panel monitoring of the microphone signal, generated anti-noise, and residual error. The results demonstrate the feasibility of real-time ANC implementation with a LabVIEW/C architecture and show that secondary-path compensation is essential for stable and effective cancellation when the loudspeaker-to-error-microphone path is included.

Contents

1	Introduction	3
2	Problem Definition	3
3	System Architecture	3
3.1	Overall Structure	3
3.2	Execution Flow	4
4	Adaptive Filtering Theory	4
4.1	FIR Controller	4
4.2	NLMS Update	4
4.3	Need for Filtered-X Adaptation	5
5	Implementation Details	5
5.1	C-Based Real-Time ANC Engine	5
5.2	Audio Buffering	5
5.3	Sample-by-Sample Processing	6
5.4	Mode Selection	6
5.5	Secondary Path Identification	6
5.6	LabVIEW Implementation	7

6	Experimental Setup	8
6.1	Hardware and Software	8
6.2	Test Signals	9
7	Results and Discussion	9
7.1	Offline Algorithm Validation and Evaluation Approach	9
7.2	Real-Time Prototype Behavior	10
7.3	Important Practical Limitations	10
8	Conclusion	10
A	Core C Implementation Excerpt	11
B	Files Included in the Final Project	11

1 Introduction

Active noise control is a signal processing technique in which an unwanted acoustic disturbance is reduced by generating a secondary sound wave with opposite phase. Unlike passive insulation, which is usually more effective at high frequencies, ANC is especially attractive for low-frequency periodic or slowly varying noise such as engine hum, fan noise, and tonal disturbances.

The goal of this project is to design, implement, and test an ANC system using adaptive filtering. The project consists of two main phases. First, the ANC algorithms were simulated offline in MATLAB to verify convergence behavior and compare NLMS and FxNLMS. Second, the final prototype was implemented using LabVIEW and a C DLL so that microphone acquisition, adaptive filtering, and front-panel visualization could run in real time.

The main contributions of the final implementation are:

- implementation of a real-time ANC engine in C,
- integration of the C engine with a LabVIEW front panel,
- support for both NLMS and FxNLMS adaptation modes,
- use of a measured or pre-estimated secondary-path model $\hat{S}(z)$,
- double-buffered microphone acquisition using the Windows Multimedia API,
- real-time display of microphone input, anti-noise output, and residual error.

2 Problem Definition

The ANC system is modeled as a feedforward adaptive control problem. A reference microphone measures a signal $x(n)$ that is correlated with the unwanted disturbance. An adaptive FIR controller $W(z)$ generates an anti-noise signal $y(n)$. This signal passes through the secondary acoustic path $S(z)$, which represents the loudspeaker, amplifier, air path, and microphone response. The residual signal at the error microphone is denoted by $e(n)$.

The objective is to minimize the residual error power

$$J(n) = E \left[e^2(n) \right], \quad (1)$$

by adapting the controller coefficients. In practice, the controller must adapt fast enough to track changes in the noise while remaining stable under real-time constraints.

3 System Architecture

3.1 Overall Structure

The final system is divided into two layers:

1. **LabVIEW layer:** provides the graphical user interface, parameter controls, waveform displays, RMS/PSD visualization, and sound output routing.
2. **C DLL layer:** performs low-level microphone acquisition, buffer management, ANC filtering, and adaptive weight updates.

The LabVIEW project contains the main VI file `ANC_main.vi`, a stripped/test VI, and the external DLL `Dynamic-Link Library.dll`. The C source file `ANC_engine.c` implements the exported functions called from LabVIEW.

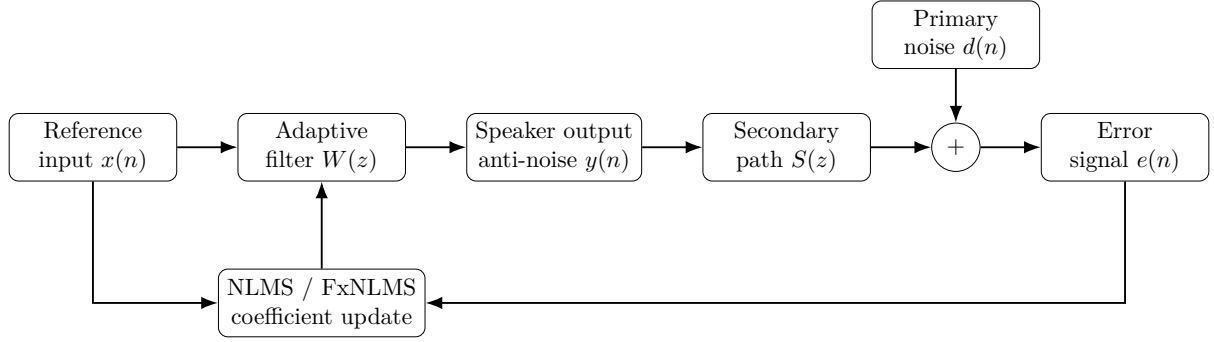


Figure 1: Block diagram of the implemented active noise control system.

3.2 Execution Flow

The real-time execution follows three stages:

1. **Initialization:** LabVIEW calls `initCAPIHardware(frameSize, sampleRate)` once before the main while-loop. This configures the microphone input stream and allocates two audio buffers.
2. **Real-time processing:** LabVIEW repeatedly calls `processANC(...)` inside the while-loop. Each call waits for a full audio frame, processes samples one by one, returns the microphone signal, residual error, and anti-noise output arrays to LabVIEW, and requeues the processed buffer.
3. **Shutdown:** LabVIEW calls `closeCAPIHardware()` when execution stops. This resets the audio stream, unprepares the buffers, closes the device, and frees memory.

This structure keeps the computationally expensive sample-by-sample filtering in C while leaving visualization and user interaction in LabVIEW.

4 Adaptive Filtering Theory

4.1 FIR Controller

The adaptive controller is an FIR filter of order L . Its coefficient vector is

$$\mathbf{w}(n) = [w_0(n) \quad w_1(n) \quad \cdots \quad w_{L-1}(n)]^T, \quad (2)$$

and the reference vector is

$$\mathbf{x}(n) = [x(n) \quad x(n-1) \quad \cdots \quad x(n-L+1)]^T. \quad (3)$$

The controller output is

$$y(n) = \mathbf{w}^T(n)\mathbf{x}(n). \quad (4)$$

4.2 NLMS Update

For the basic feedforward case, the controller is updated using NLMS:

$$\mathbf{w}(n+1) = \lambda \mathbf{w}(n) + \frac{\mu}{\delta + \mathbf{x}^T(n)\mathbf{x}(n)} e(n) \mathbf{x}(n), \quad (5)$$

where μ is the step size, δ is a small regularization constant, and λ is a leakage factor. In the final C implementation, a leakage factor of $\lambda = 0.9999$ is used to prevent unbounded coefficient growth.

4.3 Need for Filtered-X Adaptation

In a physical ANC system, the anti-noise signal does not directly appear at the error microphone. It first passes through the secondary path $S(z)$. Therefore, using the raw reference signal in the weight update can lead to incorrect gradient estimation. FxNLMS solves this problem by filtering the reference signal through an estimate of the secondary path, $\hat{S}(z)$.

The filtered reference is

$$x_f(n) = \hat{s}(n) * x(n), \quad (6)$$

where $*$ denotes convolution. The filtered reference vector is

$$\mathbf{x}_f(n) = \begin{bmatrix} x_f(n) & x_f(n-1) & \cdots & x_f(n-L+1) \end{bmatrix}^T. \quad (7)$$

The FxNLMS update is then

$$\mathbf{w}(n+1) = \lambda \mathbf{w}(n) + \frac{\mu}{\delta_f + \mathbf{x}_f^T(n) \mathbf{x}_f(n)} e(n) \mathbf{x}_f(n). \quad (8)$$

This is the main algorithm used in the final system when the secondary path is included.

5 Implementation Details

5.1 C-Based Real-Time ANC Engine

The core DSP engine is implemented in `ANC_engine.c`. The most important constants are listed in [table 1](#).

Table 1: Main parameters used in the final C implementation.

Parameter	Value	Description
<code>FILTER_ORDER</code>	64	Adaptive FIR controller length
<code>PRIMARY_DELAY</code>	16 samples	Simulated primary acoustic delay
<code>S_HAT_LEN</code>	128	Estimated secondary-path FIR length
<code>X_HISTORY_LEN</code>	256	Reference history buffer length
Audio format	16-bit PCM mono	Microphone input format
Nominal sample rate	44.1 kHz	Audio sampling frequency
Leakage factor	0.9999	Weight leakage in NLMS/FxNLMS

At 44.1 kHz, the 16-sample primary delay corresponds to

$$\tau_p = \frac{16}{44100} \approx 0.363 \text{ ms}. \quad (9)$$

Using a sound speed of approximately 343 m/s, this delay corresponds to an acoustic distance of approximately

$$d_p = 343 \times \tau_p \approx 0.124 \text{ m}. \quad (10)$$

This is consistent with a short microphone-to-earcup or microphone-to-loudspeaker spacing.

5.2 Audio Buffering

The implementation uses two raw audio buffers, `rawBufferA` and `rawBufferB`. While one buffer is filled by the audio driver, the other buffer is processed by the ANC engine. This ping-pong buffering structure reduces the probability of data loss and allows continuous streaming.

In each processing call, the code waits until the selected buffer is marked as complete using the `WHDR_DONE` flag. After processing, the same buffer is requeued with `waveInAddBuffer`. This creates a continuous loop of acquisition and processing.

5.3 Sample-by-Sample Processing

For every sample in a frame, the following operations are executed:

1. Normalize the 16-bit microphone sample to the range $[-1, 1]$.
2. Update the reference history buffer $\mathbf{x}(n)$.
3. Compute the filtered-X sample using $\hat{S}(z)$.
4. Update the filtered-X history buffer $\mathbf{x}_f(n)$.
5. Compute the anti-noise output $y(n)$ using the adaptive FIR filter.
6. Simulate the primary delay to obtain $d(n)$.
7. Simulate the secondary-path output $y_s(n)$ using $\hat{S}(z)$.
8. Compute the residual error $e(n) = d(n) - y_s(n)$.
9. Update the adaptive weights using either NLMS or FxNLMS.

The output arrays passed back to LabVIEW are the microphone display signal, the residual error signal, and the generated anti-noise signal.

5.4 Mode Selection

The C function `processANC` includes a `caseSelection` argument:

- `caseSelection = 1`: direct NLMS update using the raw reference vector.
- `caseSelection = 2`: FxNLMS update using the filtered reference vector.

This allows the same front panel to compare the basic NLMS controller and the secondary-path-aware FxNLMS controller.

5.5 Secondary Path Identification

For the FxNLMS algorithm, the secondary path must be estimated before real-time operation. In this project, the secondary path represents the acoustic and electronic path from the generated anti-noise signal to the error microphone. This path includes the DAC, amplifier, headset speaker, physical air gap inside the earcup, and the microphone response.

Since the available headset setup did not include a permanently mounted error microphone inside the earcup, online secondary path identification could not be performed during real-time operation. Therefore, an offline calibration procedure was carried out in MATLAB before running the real-time ANC system.

The secondary path identification procedure was performed as follows:

1. A known White Gaussian Noise signal was generated in MATLAB and played through the headset speaker.
2. A microphone was placed close to the speaker driver inside the earcup to record how the noise changed as it traveled through the DAC, amplifier, speaker, and acoustic air gap.
3. The original white noise signal was used as the input of an offline 128-tap NLMS adaptive filter, while the recorded earcup microphone signal was used as the desired target signal.
4. Since white noise has a broad and approximately flat frequency spectrum, the adaptive filter was able to learn the physical behavior of the secondary acoustic path.

5. After convergence, the final 128 adaptive filter coefficients were extracted as the estimated secondary path impulse response.
6. These 128 coefficients were exported from MATLAB and hardcoded into the C DLL array used for the filtered-X convolution during runtime.

Figure 2 shows the calibrated 128-tap secondary path impulse response obtained using the offline MATLAB calibration. The response has a strong initial component followed by a decaying oscillatory behavior, which is consistent with the combined effect of the speaker, microphone, and acoustic coupling inside the headset earcup.

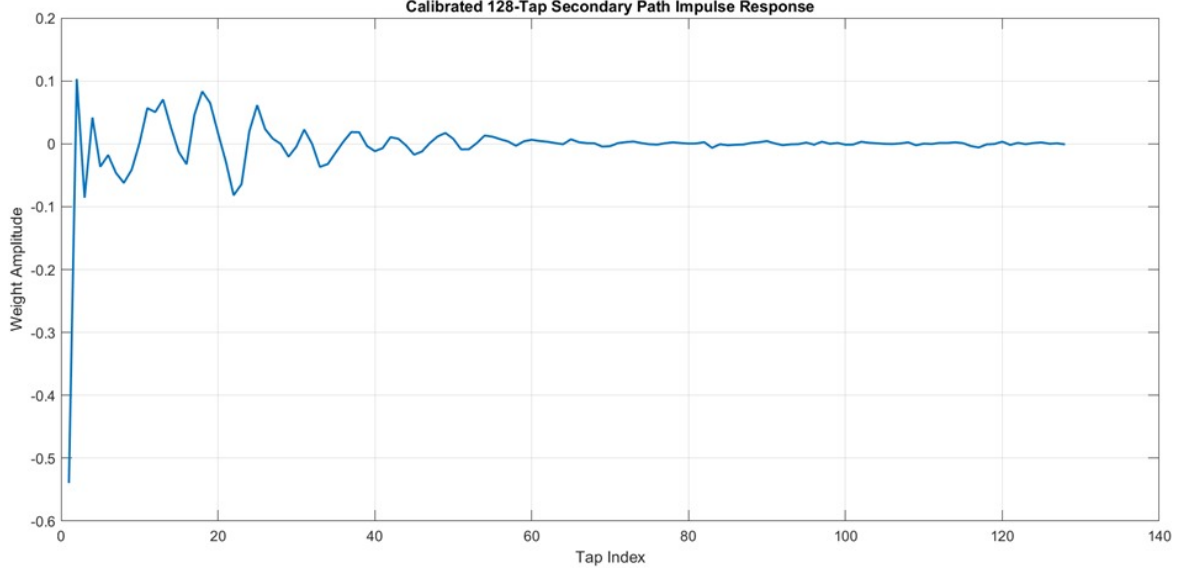


Figure 2: Calibrated 128-tap secondary path impulse response obtained using offline NLMS identification in MATLAB.

This fixed setup is not ideal. In an ideal deployment, a secondary error microphone would be permanently mounted inside the earcup so that the secondary path model could be updated during runtime. In our prototype, we were limited to a single inline headset microphone on the cable; therefore, online secondary path identification was not feasible. Instead, we used this pre-calibrated snapshot and assumed that the physical acoustic path remained approximately constant during operation.

5.6 LabVIEW Implementation

LabVIEW was used as the graphical interface and integration environment of the project. The front panel allows the user to start and stop the ANC system, select parameters, and observe the microphone input, generated anti-noise, and residual error signals in real time. The LabVIEW block diagram connects the graphical interface to the C-based ANC engine through Call Library Function Nodes that call the compiled DLL functions.

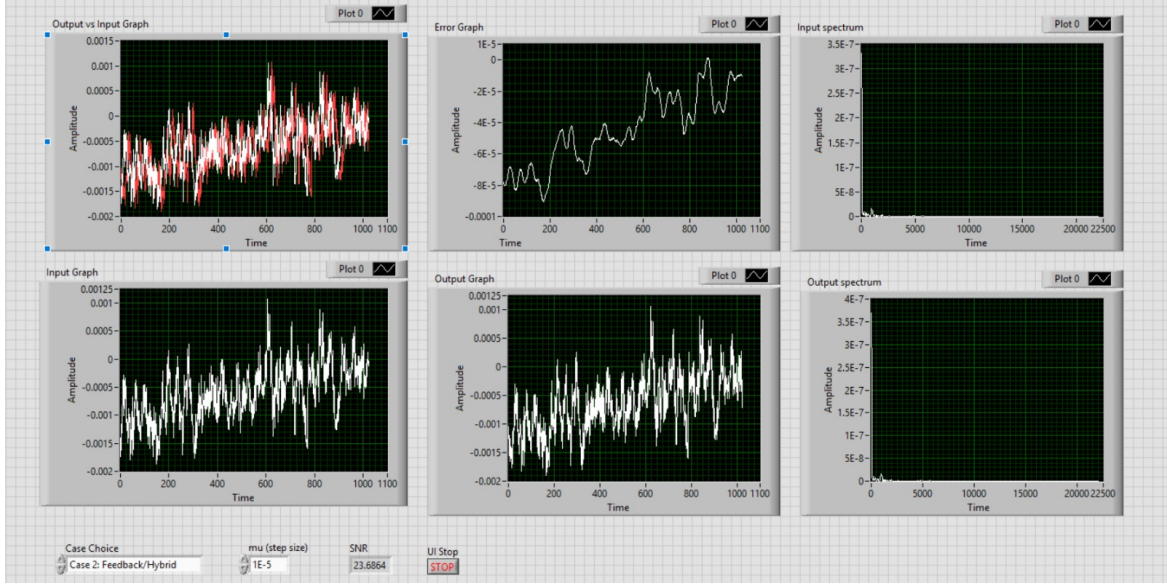


Figure 3: LabVIEW front panel of the real-time ANC system.

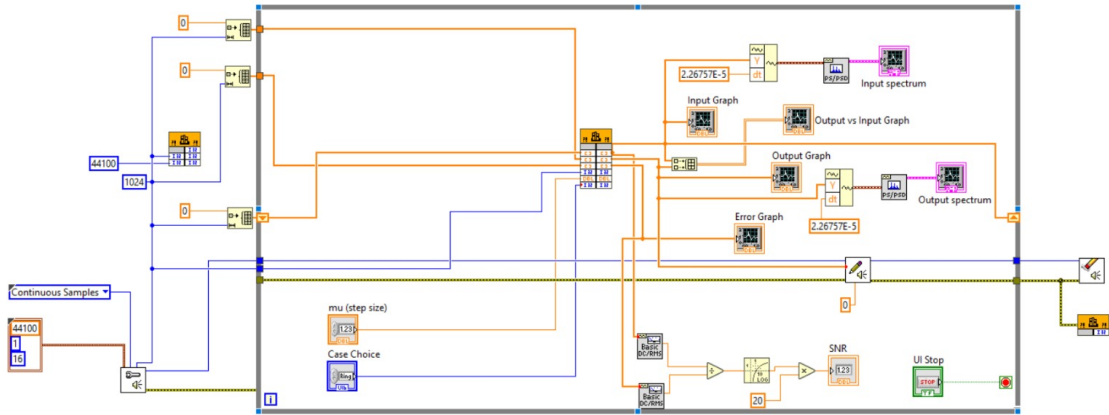


Figure 4: LabVIEW block diagram used to call the C DLL and control the ANC process.

6 Experimental Setup

6.1 Hardware and Software

The final prototype was tested on a Windows computer using LabVIEW and a Visual Studio C project that builds the ANC engine DLL. The microphone stream is acquired using the Windows Multimedia API. The DLL is called from LabVIEW through Call Library Function Nodes.

Table 2: Experimental setup summary. Replace the placeholders with exact hardware names if needed.

Component	Description
Computer	Windows laptop/desktop running LabVIEW
Programming tools	LabVIEW, Visual Studio, C DLL
Audio API	Windows Multimedia API, <code>waveIn</code> interface
Input	Mono microphone input
Output	LabVIEW sound output / headphone or loudspeaker output
Adaptive algorithms	NLMS and FxNLMS
Secondary path	128-tap FIR estimate $\hat{S}(z)$

6.2 Test Signals

During the offline validation phase, several noise profiles were considered, including single-tone noise, two-tone noise, band-limited noise, white noise, engine-like noise, and amplitude-modulated low-frequency hum. These signals are useful because ANC is expected to perform best for coherent low-frequency components and more poorly for broadband uncorrelated noise.

For the final real-time implementation, the microphone signal is acquired directly from the hardware. Therefore, the effective test signal depends on the acoustic environment, speaker placement, microphone placement, and room reflections.

Note on the physical evaluation. In the final headset-based prototype, a separate physical error microphone was not available inside the earcup. Therefore, the real-time demonstration focuses on observing the input, generated anti-noise, and model-based residual signal rather than reporting a fully physical closed-loop SNR measurement.

7 Results and Discussion

7.1 Offline Algorithm Validation and Evaluation Approach

The offline MATLAB stage verified that the adaptive filter converges for coherent low-frequency disturbances. In the feedforward case, NLMS reduces predictable narrowband noise, while in the secondary-path case FxNLMS provides more reliable convergence because the adaptation uses the filtered reference signal. The strongest cancellation is expected for tonal and engine-like noise, while white noise is harder to suppress because it is broadband and less predictable.

A numerical SNR table was not reported for the final real-time headset demonstration because the setup did not include an independent secondary error microphone inside the earcup. In the implemented C code, the residual error used for display and adaptation is model-based: the primary path is represented by a delay, and the secondary path output is simulated using the calibrated $\hat{S}(z)$. Therefore, an SNR value computed directly from this model-based residual would mainly reflect the assumed model and can become unrealistically high, especially for single-tone and white-noise test cases. For this reason, the final evaluation is presented qualitatively and through implementation evidence rather than as a misleading physical SNR measurement.

Table 3: Qualitative evaluation summary of the ANC implementation.

Test or observation	Algorithm/mode	Interpretation
Offline single-tone noise	NLMS / FxNLMS	Fast convergence is expected because the disturbance is narrowband and predictable.
Offline white noise	NLMS / FxNLMS	Broadband cancellation is more difficult; model-based SNR values may appear artificially high and should not be interpreted as physical acoustic attenuation.
Secondary-path calibration	Offline NLMS	The 128-tap impulse response was estimated and used as the fixed $\hat{S}(z)$ model for FxNLMS.
Real-time LabVIEW/C operation	FxNLMS mode	The system demonstrates real-time microphone acquisition, DLL communication, filtered-X processing, and waveform visualization.
Physical ANC performance	Prototype limitation	A separate earcup error microphone is needed for reliable physical SNR or dB attenuation measurement.

7.2 Real-Time Prototype Behavior

The LabVIEW/C prototype confirms that the adaptive filtering loop can be executed in real time. The main advantage of moving the DSP code to C is that sample-by-sample filtering and weight updates are handled efficiently, while LabVIEW remains responsible for visualization and user controls.

The FxNLMS mode is more physically meaningful than the direct NLMS mode because it accounts for the secondary path between the generated anti-noise signal and the error microphone. Without this compensation, the adaptive filter may update in the wrong direction when the secondary path introduces delay, phase shift, or spectral coloration.

7.3 Important Practical Limitations

Several practical factors affect real-time ANC performance:

- **Latency:** Any delay in acquisition, processing, LabVIEW output, or speaker playback reduces cancellation performance, especially at higher frequencies.
- **Secondary-path mismatch:** FxNLMS assumes that $\hat{S}(z)$ is close to the true secondary path. A poor estimate can slow convergence or cause instability.
- **Microphone and speaker placement:** Small changes in geometry can alter the acoustic paths and therefore change the optimal filter.
- **Broadband noise:** Random broadband noise is more difficult to cancel than tonal or engine-like noise.
- **Frame size:** Larger frames reduce overhead but increase latency. Smaller frames reduce latency but increase CPU overhead.

8 Conclusion

This project developed a complete active noise control system from algorithm design to real-time implementation. The MATLAB stage showed the expected behavior of NLMS and FxNLMS for different noise types, while the final LabVIEW/C prototype demonstrated real-time acquisition,

adaptive filtering, and visualization. The final C implementation uses a 64-tap adaptive controller, a 128-tap secondary-path model, double-buffered microphone acquisition, and selectable NLMS/FxNLMS update rules.

The results support the main conclusion that secondary-path compensation is necessary for practical ANC. Direct NLMS is useful as a baseline, but FxNLMS is better suited for real acoustic systems because it adapts using a reference signal filtered by $\hat{S}(z)$. Since the prototype did not include a separate physical error microphone inside the earcup, the real-time results should be interpreted as a functional demonstration of the LabVIEW/C ANC pipeline rather than as a complete physical SNR measurement. Future improvements could include online secondary-path identification, lower-latency audio output, a measured physical error microphone, and systematic evaluation under controlled acoustic conditions.

A Core C Implementation Excerpt

```

1  if (caseSelection == 1) {
2      double norm = 1e-6;
3      for (int j = 0; j < FILTER_ORDER; j++) {
4          norm += x_history[j] * x_history[j];
5      }
6      for (int j = 0; j < FILTER_ORDER; j++) {
7          w_weights[j] = (0.9999 * w_weights[j])
8                          + (mu / norm) * error * x_history[j];
9      }
10 }
11 else if (caseSelection == 2) {
12     double norm_fx = 1e-1;
13     for (int j = 0; j < FILTER_ORDER; j++) {
14         norm_fx += x_filt_history[j] * x_filt_history[j];
15     }
16     for (int j = 0; j < FILTER_ORDER; j++) {
17         w_weights[j] = (0.9999 * w_weights[j])
18                         + (mu / norm_fx) * error * x_filt_history[j];
19     }
20 }

```

Listing 1: Simplified weight update structure used in the final ANC engine.

B Files Included in the Final Project

Table 4: Main submitted project files.

File	Purpose
ANC_main.vi	Main LabVIEW front panel and execution loop
ANC_stripped.vi	Simplified/test LabVIEW VI
ANC_project.lvproj	LabVIEW project file
ANC_engine.c	C source code for the ANC DSP engine
Dynamic-Link Library.dll	Compiled DLL called by LabVIEW
EE433_ANC_Engine.vcxproj	Visual Studio C/C++ project file

References

- [1] S. M. Kuo and D. R. Morgan, *Active Noise Control Systems: Algorithms and DSP Implementations*. Wiley, 1996.
- [2] S. Haykin, *Adaptive Filter Theory*, 4th ed. Prentice Hall, 2002.
- [3] S. J. Elliott, *Signal Processing for Active Control*. Academic Press, 2001.
- [4] B. Widrow and S. D. Stearns, *Adaptive Signal Processing*. Prentice Hall, 1985.